The Last Dino

Priya Patel and Heather Monteson

Final State:

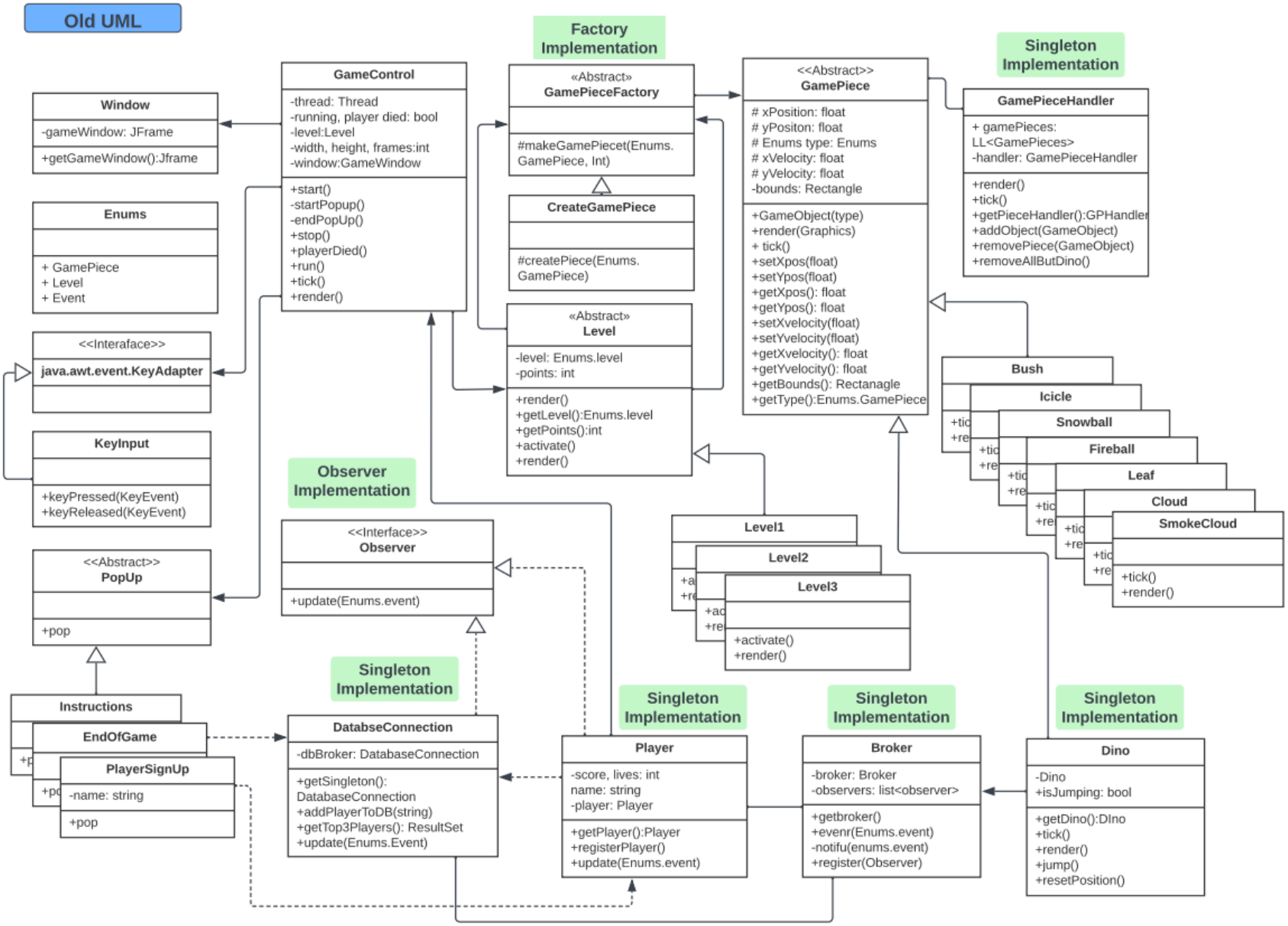
All functional requirements that were laid out at the beginning of project 5 have been implemented. However, to create the illusion of the dino running forward in the game we selected not to move the background, and instead chose to move specific game pieces as the player's dino was running. We also did not initially consider how to switch between levels, and have since implemented level switching by setting each level on a timer. The timer and current level number have also been made visible to the player in the Graphical User Interface (GUI) so they can keep track of where they are in the game.

Third Party Code, Frameworks, Tools and Resources:

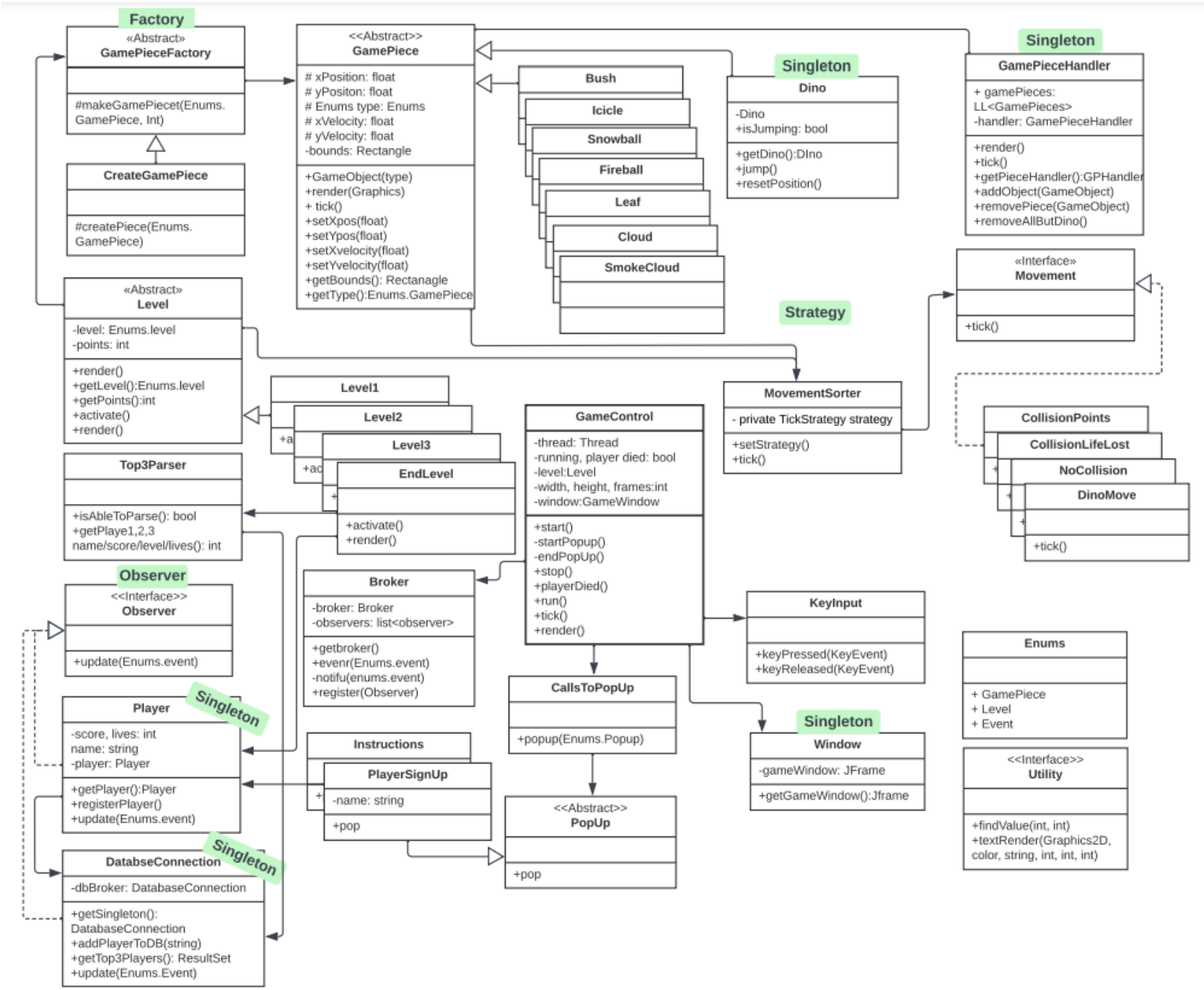
1) Open Source Code and Tutorials:
    a) Database set up: https://www.youtube.com/watch?v=e8g9eNnFpHQ used to learn how to make the initial connection to the MySQL database.
    b) General backend setup: https://www.youtube.com/watch?v=1gir2R7G9ws used the open source code for the game loop in the game controller. Also utilized tutorial for ideas in basic setup for our game piece handler and understanding how the rectangle class framework could be implemented for our collisions.
    c) Text rendering: used for setting up text rendering method used by multiple classes from the utility interface http://www.java2s.com/Code/Java/2D-Graphics-GUI/Draw2DText.htm
2) Frameworks:
    a) Extended KeyAdapter from The Abstract Window Toolkit (AWT): used the framework and implemented the key pressed and key released methods with original code to control the game pieces. https://docs.oracle.com/javase/7/docs/api/java/awt/event/KeyEvent.html#VK_W
    b) Extended Canvas: used to create all game windows/pop ups but built with original code. Utilized documentation https://docs.oracle.com/javase/7/docs/api/java/awt/Canvas.html and https://docs.oracle.com/javase/tutorial/uiswing/components/frame.html
    c) Rectangle: used the framework for tracking collisions between dino and other game pieces https://www.youtube.com/watch?v=3paMFMwVfWU&t=600s
3) Additional Tools and APIs:

- Swing
- AWT
- JavaFx
- Thread
- Serial (requires Java 14+ for serial in build base)
- Util
- SQL and using JDBC for database connection
- Canva: design instructions/end graphics

Initial UML Diagram:



Final UML Diagram:



Changes from Initial and Final UML Diagrams:

From the initial to the final UML not a lot of large changes were made, however, we did add in a strategy pattern to pull out the variations of our tick algorithms. Our strategy pattern consists of 4 different movement behaviors that extend from an interface called Movement. We then have a class called MovementSorter which encapsulates these behaviors and provides a way to set and call them. We also removed the command line pattern since we felt it would make the code too complex and just sending signals with the key input would suffice. Additionally, we removed a lot of methods from the game piece to include only those necessary for controlling game piece movement and location updates.

Process Statement:

1) Planning and functional requirements: Starting the project we included a lot of detail in what we wanted to accomplish and planned out a lot of the functional elements of the overall design early on. This helped a lot with breaking down what we needed to work on and section out what elements could be developed separately. Along with the requirements it also was good to have an idea of what the constraints were so that we could have a better understanding of the features we wanted and ones that we did not.
2) Pattern planning: Because we were creating something in Java that neither of us had any experience in, it was difficult to anticipate what patterns we might need or how to incorporate them in the planning stages. However, we did find that singleton patterns were really useful for tracking specific objects and their data through the entire game play and the factory method allowed us to encapsulate some more complex and changing algorithms we needed to create specific game pieces to set their starting location. Additionally, the observer pattern was really useful for tracking and updating graphics for various events that occurred during game play like when the user eats leaves or collides with an obstacle and loses a life.
3) Graphics: Neither of us had previous experience with rendering graphics, so it took a lot of trial and error to finally get something working. Once we initially got graphics laid out in the window, it was easy to add additional graphics. However, the next problem we faced was automating the creation of certain game pieces in a way that was thoughtful to the game. Doing this without much previous knowledge of the graphics class we utilized was difficult. The nice part of this game was that as soon as you fixed a problem with one game piece, you fixed the problem for the other 10 game pieces.

Github

https://github.com/heatherMonteson/TheLastDino